



Advanced Character Animation and Locomotion

A practical setup guide for 1D and 2D Blend Spaces, movement states, root motion, layered actions, events, and runtime debugging

Engine workflow

Character Editor -> Animation Graph -> Play -> Export

Updated for the directional Blend Space and locomotion parameter system

1. What the locomotion system provides

The Character Editor now owns the character animation graph and previews it on the skeletal model selected by the character asset. The same graph is serialized into scenes, used by editor Play, and exported to the standalone player.

- **1D Blend Spaces** for idle, walk, and run on a Speed axis.
- **2D directional Blend Spaces** using Speed and Direction, with up to four nearby samples blended continuously.
- **Cycle synchronization** so clips with different durations retain a common locomotion phase.
- **Weighted root motion** using the same active Blend Space samples.
- **Movement parameters** for acceleration, deceleration, turn rate, stopping, grounded, falling, and vertical speed.
- **Transition graphs** for start, stop, pivot, turn-in-place, jump, fall, and land states.
- **Layered actions** for attacks and magic while the lower body continues locomotion.
- **Animation events** for footsteps, damage windows, audio, particles, and scripted effects.
- **Live debugging** showing the active state and weight of every active Blend Space sample.

Important model preparation

Use clips authored for the same skeleton and forward axis. In-place locomotion is the easiest starting point. Root-motion clips may be used, but enable root motion only on states whose translation should drive the character. Clip names such as Idle, Walk_Forward, Run_Forward, Strafe_Left, and Strafe_Right make setup much easier.

The system performs runtime pose blending. It does not repair animation retargeting errors; retarget incompatible clips before import.

2. Create the character asset

1. Open the Character Editor and create or load a .3dgcharacter asset.
2. In Model, choose the skeletal model. Confirm that the preview faces the camera and plays a clip.
3. In Collision, enable a capsule collider, fit it around the body, and use Player or Enemy collision channels.
4. Enable Player Controller for a player character. Set walk speed, run speed, jump speed, step height, and maximum slope.
5. Open Animation Graph. The graph now edits this character asset directly; it does not depend on a hierarchy selection.

Recommended movement values

Setting	Starting value	Purpose
Walk speed	2.0-4.0	Normal locomotion and walk sample position
Run speed	5.0-8.0	Sprint locomotion and run sample position
Step height	0.30-0.40	Allows normal stairs and small ledges
Maximum slope	45-55 degrees	Rejects surfaces that should behave as walls
Blend fade	0.12-0.25 s	Smooth state changes without sluggish controls

3. Fast 1D locomotion setup

Click **Create 1D Locomotion**. The editor creates a Locomotion state driven by Speed. Assign Idle, Walk, and Run clips and use values matching the controller movement speeds.

Sample	Clip	Axis value
Idle	Idle loop	0.0
Walk	Walk forward loop	2.0 or authored walk speed
Run	Run forward loop	6.0 or authored run speed

Enable **Synchronize Animation Cycles**. Move the live Speed parameter through the sample range and watch the sample weight bars. At Speed 1 between samples 0 and 2, Idle and Walk should each be close to 50 percent.

4. 2D directional locomotion

Click **Create 2D Directional Locomotion**. This creates the required movement parameters and a directional state using Speed for the first axis and Direction for the second. Direction is measured in degrees relative to character forward.

Movement	Speed	Direction	Typical clip
Idle	0	0	Idle
Forward walk	2	0	Walk forward
Right strafe	2	90	Strafe right
Left strafe	2	-90	Strafe left
Backward	2	180	Walk backward
Forward run	6	0	Run forward

Direction wraps across -180 and 180 degrees, preventing a discontinuity when moving backward. The runtime selects the nearest directional samples and normalizes their weights. Exact sample positions resolve to one clip.

For combat locomotion, keep the character facing the target and let Direction select forward, backward, and strafe clips. For free locomotion, the controller can rotate toward movement and Direction will normally remain near zero.

Suggested sample layout

Speed	-90 Left	0 Forward	90 Right	180 Back
0		Idle		
2	Walk L	Walk F	Walk R	Walk B
6	Run L	Run F	Run R	Run B

5. Build the advanced locomotion state graph

Blend Spaces cover continuous locomotion. Short authored states handle deliberate starts, stops, pivots, turns, and airborne motion. Add states only when matching clips exist.

State	Loop	Condition to enter	Condition to leave
Start	No	IsMoving == true	Exit time near 0.85 -> Locomotion
Stop	No	IsStopping == true	Exit time near 0.90 -> Locomotion
Pivot Left/Right	No	Direction changes sharply	Exit time near 0.85 -> Locomotion
Turn Left	No	TurnRate < negative threshold	Exit time near 0.90 -> Locomotion
Turn Right	No	TurnRate >= positive threshold	Exit time near 0.90 -> Locomotion
Jump	No	IsGrounded == false	VerticalSpeed < 0 -> Fall
Fall	Yes	IsFalling == true	IsGrounded == true -> Land
Land	No	IsGrounded == true	Exit time near 0.80 -> Locomotion

Use transition priority so airborne transitions beat stop or pivot transitions. Enable Interrupt Blend for urgent transitions such as falling or hit reactions. Keep ordinary locomotion fades short; use a longer fade only where the authored poses differ greatly.

Acceleration is signed speed change per second. Deceleration is positive while slowing. IsStopping pulses when movement reaches zero. These values are published automatically for both editor Play and exported player builds.

6. Synchronization, root motion, and foot stability

Enable Synchronize Animation Cycles on locomotion Blend Spaces. Each sample is evaluated at the same normalized phase, even if its clip duration differs. This greatly reduces mismatched left/right steps while blending walk and run.

Root motion

Enable Root Motion on a state only when the animation should move the entity. Blend Space root translation is computed from every active sample using its weight, then applied in character orientation. Keep capsule collision active so the character remains constrained by the level. Do not combine authored forward root motion with controller translation unless deliberate.

Foot sliding checklist

- Set sample Speed values close to each clip's authored travel speed.
- Keep cycle synchronization enabled.
- Use in-place clips when controller movement is authoritative.
- Use root-motion clips consistently within one state.
- Avoid blending clips with opposite foot phases; re-time them in the DCC tool when necessary.
- Tune walk/run controller speeds after watching the feet, not only by numeric convention.

Grounding and stairs

The character capsule and AI grounded movement own collision, gravity, slope rejection, and step climbing. Animation reads IsGrounded, IsFalling, and VerticalSpeed. Foot IK and stride/orientation warping require rig-specific bone chains and are best added as a later skeletal post-process profile; the current system provides the movement parameters and synchronized poses needed by that pass.

7. Attacks, magic, additive motion, and events

Use action profiles for attacks, spell casts, hit reactions, and staff motion. A full-body action blocks movement until it completes, matching montage behavior. A profile with a bone mask remains layered over locomotion, allowing the legs to keep moving.

Example	Mask root	Behavior
Staff attack	Spine or upper chest	Upper body attacks while legs remain in locomotion
Full-body finisher	Empty mask	Movement locked until the action completes
Aim offset / recoil	Spine	Small layered upper-body correction
Hit reaction	Spine or empty	Partial reaction or full-body interruption

Add animation events at precise times. Useful names include Footstep_L, Footstep_R, AttackWindowOpen, AttackWindowClose, StaffCast, SpawnFire, and Land. Scripts can listen for these events to play audio, apply damage, or spawn particle systems.

Animation curves can be represented with continuously updated graph parameters and timed events. For example, a script may update AimPitch every frame while an event triggers the instant a fire projectile should spawn.

8. Preview and debug

- Load the correct skeletal model in the Character Editor.
- Press Play in the preview and change Speed and Direction.
- Confirm the Active State label.
- Watch Active Blend Samples; weights should sum to 1.0.
- Test exact sample positions, midpoints, backward wrap, and zero speed.
- Test start/stop and airborne transitions using the live bool and float parameters.
- Run editor Play and compare against the Character Editor preview.
- Export and run the standalone player; the same Speed, Direction, acceleration, turn, and grounded parameters are published there.

Common problems

Symptom	Likely cause	Fix
Feet slide	Axis speed differs from authored clip speed	Adjust sample position or controller speed
Wrong strafe	Direction sign or clip assignment reversed	Swap left/right or change +/-90 sample
Backward blend pops	No 180-degree sample	Add backward samples; direction wrapping is automatic
Walk/run feet disagree	Cycles are out of phase	Enable synchronization and re-time source clips
State never exits	No valid transition or exit time	Check parameter name, compare mode, priority, and exit time
Movement continues during attack	Action has a bone mask	Use an empty mask for a montage-style full-body action

9. Complete example: staff-wielding player

Locomotion state: 2D Blend Space with Idle at (0,0), Walk Forward at (2,0), Walk Left at (2,-90), Walk Right at (2,90), Walk Back at (2,180), and Run Forward at (6,0). Synchronization is enabled.

Airborne states: Jump -> Fall -> Land, driven by IsGrounded, IsFalling, and VerticalSpeed. Airborne transitions have higher priority than stop and pivot transitions.

Turn states: Turn Left and Turn Right use TurnRate thresholds while Speed is approximately zero. They return to Locomotion after exit time.

Attack action: StaffAttack uses an upper-spine mask so movement can continue. StaffFinisher has no mask and therefore locks movement until completion.

Magic event: SpawnFire is placed at the frame where the staff points forward. The script receives the event and spawns the fire particle asset at the staff socket or helper object.

Verification checklist

- Idle, walk, run, strafe, and backward movement blend without pose popping.
- Speed 0 settles completely into Idle.
- Direction changes continuously and wraps behind the character.
- Walk and run feet remain in phase.
- Jump and fall override ground locomotion.
- Full-body actions stop controller movement; masked actions do not.
- Editor Play and exported player produce the same graph behavior.
- Character asset and scene reload without losing Blend Space samples.

Recommended next rig-specific pass

After the core graph is stable, add a skeleton profile for foot IK, pelvis compensation, stride warping, and orientation warping. These features depend on named hip, knee, ankle, and toe bones plus per-foot ground traces, so they should be configured per skeleton rather than guessed globally. Motion matching is a separate database-driven system and should follow reliable locomotion tagging, curves, and trajectory sampling.

End of guide